

برنامج إدارة درجات الطلاب

```
def manage_student_grades():
    # قائمة من الصفوف (اسم، درجات)
    students = [
        ('Ahmed', [85, 92, 78, 90]),
        ('Sara', [88, 95, 82, 87]),
        ('Omar', [79, 83, 91, 85])
    ]

    # حساب المعدلات
    results = []
    for name, grades in students:
        average = sum(grades) / len(grades)
        results.append((average, name, grades))

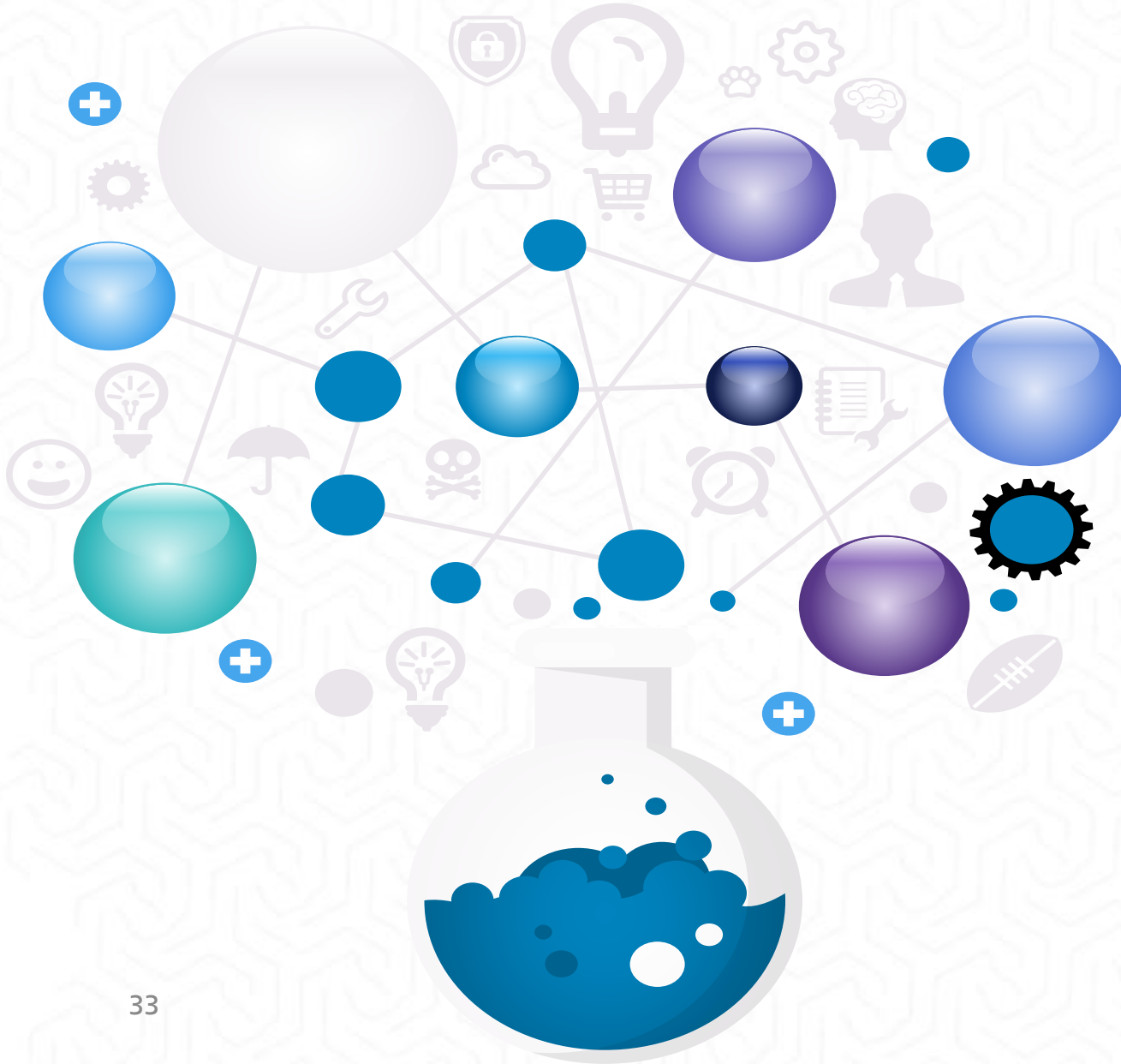
    # ترتيب حسب المعدل (تنازلي)
    results.sort(reverse=True)

    print("ترتيب الطلاب حسب المعدل:")
    print("-" * 40)

    for i, (avg, name, grades) in enumerate(results, 1):
        print(f"{i}. {name}: {avg:.1f}")
        print(f"   الدرجات: {grades}")

    return results

# تشغيل البرنامج
manage_student_grades()
```



القوائم مرنة وقابلة للتعديل، مناسبة للبيانات المتغيرة

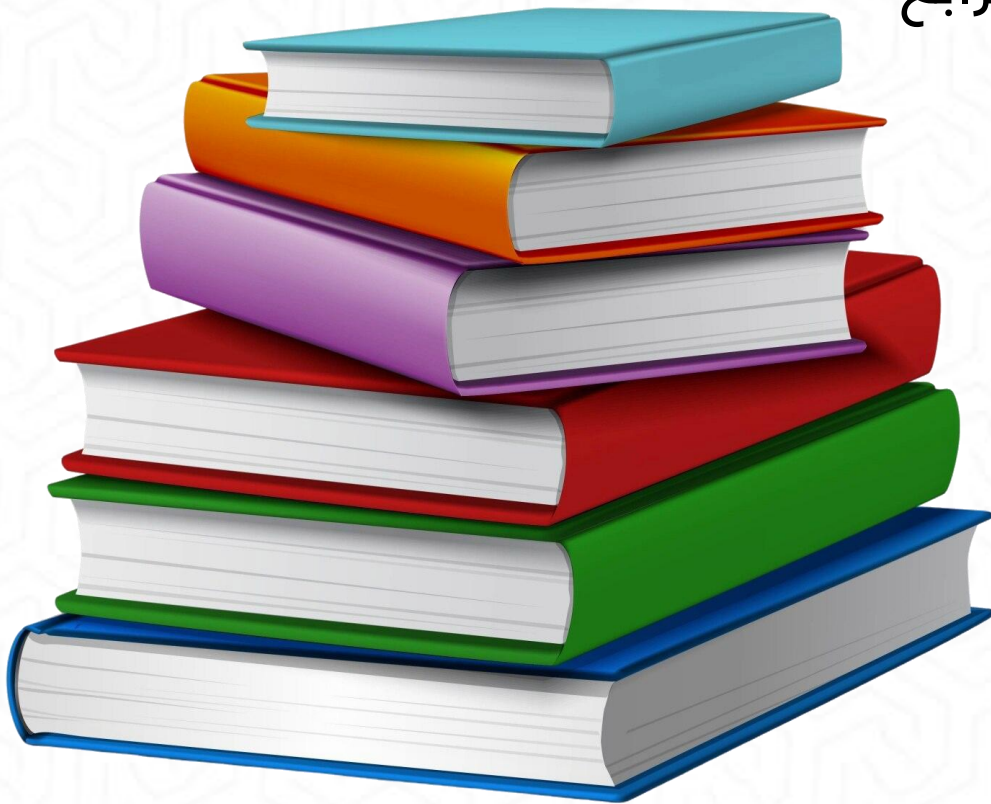
الصفوف ثابتة وآمنة، مناسبة للبيانات الثابتة

كلاهما يدعم الفهرسة، التجزئة، والحلقات

فهم المراجع مهم لتجنب الأخطاء الشائعة

الاختيار الصحيح بين القوائم والصفوف يحسن من كفاءة البرنامج

Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: : PYT103

الموضوع:العوامل والتعبيرات



الموضوع الثالث: العوامل والتعبيرات

الجمهورية التونسية

الأهداف

في نهاية هذا الدرس ستكون قادرًا على أن:

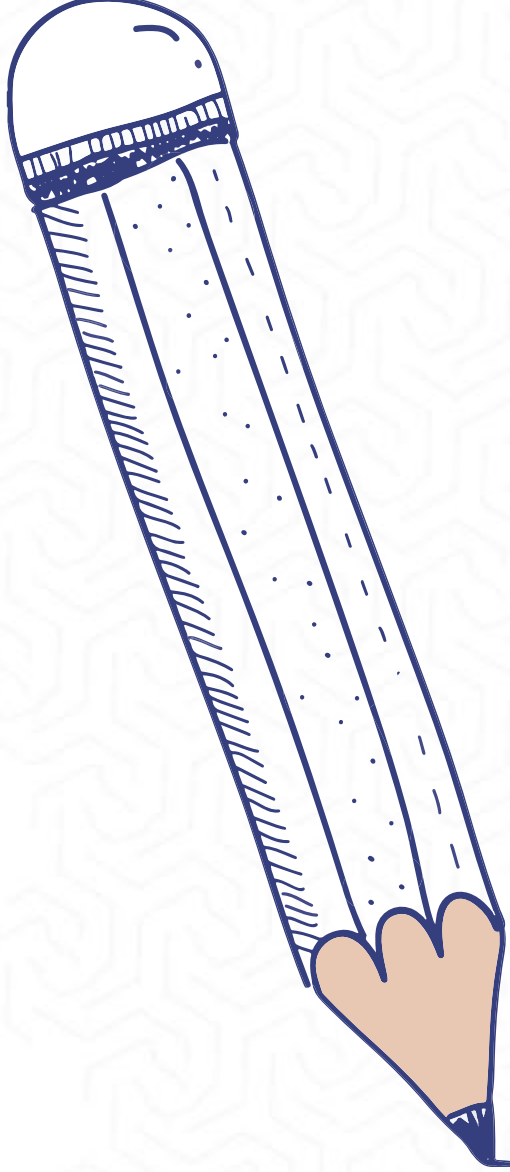
- ✓ تعريف العوامل (Operators) والمعاملات (Operands) والتمييز بينهما.
- ✓ استخدام العوامل الحسابية الأساسية (+, -, *, /, //, %, **) في أمثلة عملية.
- ✓ تفسير سلوك القسمة في بايثون والتمييز بين القسمة العشرية (/), والقسمة الصحيحة (//), وباقي القسمة (%).
- ✓ تطبيق قاعدة تراتبية العمليات (PEMDAS) وفهم الأسبقيات والارتباط من اليسار لليمين.
- ✓ بناء تعابير صحيحة وتوظيفها في السطر التفاعلي والملفات البرمجية مع التمييز بين "تعليمة التعبير" و "تعليمة الإسناد".
- ✓ إجراء العمليات على السلاسل النصية باستخدام العاملين (+) و (*).



وصف الدرس

في هذا الدرس سنتناول:

- يُعرّف هذا الدرس الطالب بالعوامل والتعابير في بايثون باعتبارها "لغة الحساب" داخل البرنامج.
- نبدأ بتوضيح معنى العامل والمعاملات، ثم نستعرض أهم العوامل الحسابية وكيفية عملها على الأعداد والسلاسل النصية.
- بعد ذلك نطبّق قواعد ترابطية العمليات لضمان كتابة تعابير دقيقة، ونبيّن الفرق بين التقييم في الوضع التفاعلي وبين تضمين التعابير داخل ملف برمجي، مع التركيز على الاستخدام السليم لتعليمة التعبير (expression statement) وتعليمة الإسناد (assignment). يتضمن الدرس أمثلة تطبيقية وتمارين قصيرة لتعزيز الفهم.



(١) العامل Operator: رمز خاص يرتبط بعملية (جمع، طرح، ضرب ...).

(٢) المعاملات Operands: القيم التي تُطبّق عليها العملية.

(٣) التعبير Expression: تركيب من قيم/متغيّرات وعوامل ينتج عنه قيمة.

(٤) تقييم التعبير Evaluate: تنفيذ خطوات العمليات لإنتاج النتيجة.

أهم العوامل الحسابية:

+ الجمع، - الطرح، * الضرب، / القسمة العشرية، ** الرفع إلى قوة، // القسمة الصحيحة (تقريب للأدنى)، % باقي القسمة. مع أمثلة:

```
20 + 32
hour*60 + minute
minute / 60
5**2
(5+9)*(15-7)
```



(١) تعليمية التعبير: كتابة تعبير مستقل (في التفاعلي) تعرض نتيجته مباشرة.

(٢) تعليمية الإسناد: تخزين نتيجة التعبير في متغير: $x = 5 * 3$

(٣) في ملف برمجي: التعبير وحده لا يظهر شيئاً ما لم نطبعه أو نُسنده.

مثال (تفاعلي):

```
>>1 + 1  
2
```



الموضوع الثالث | القسمة

عملية القسمة باستخدام / تعيد دائمًا عددًا عشريًا float حتى عند قسمة أعداد صحيحة.



minute = 59

x = minute / 60 # x = 0.9833333333333333



استخدام % يعطي الباقي عند قسمة عدد صحيح على آخر.



quotient = 7 // 3 # quotient=2
remainder = 7 % 3 # remainder=1



عملية القسمة باستخدام // تُعيد الجزء الصحيح (تقريبًا إلى الأدنى).



minute = 59

x = minute // 60 # x=0



استخدم // عندما تحتاج عدد الدقائق/الساعات الكامل بدون الكسور.

• القابلية للقسمة: إذا كان $X \% Y == 0$ ، ف X يقبل القسمة على Y .

$n = 42$

زوجي # $n \% 2 == 0$

يقبل القسمة على 7 # $n \% 7 == 0$



استخراج الأرقام الأخيرة:

آخر رقم # $x \% 10$

آخر رقمين # $x \% 100$

الناتج رقم 5 # $115 \% 10$

الناتج رقم 15 # $115 \% 100$



الموضوع الثالث | تراتبية العمليات PEMDAS

- الأقواس ثم الأس **, ثم الضرب/القسمة، ثم الجمع/الطرح. الأقواس تغيّر الترتيب، وتسهّل القراءة.

$(1 - 3) * 2 \# -4$ = ناتج العملية
 $(5 - 2) ** (1+1) \# 9$ = ناتج العملية

مثال يوضح الأسبقية:

$$2 * 3 - 1$$

النتيجة ليست $(3-1)^*$ #٢

$$1 + 1/\varepsilon$$

تنفّذ من اليسار لليمين ضمن نفس الأسبقية#

استخدم الأقواس لتحديد المقصود بدقة.



الموضوع الثالث | الارتباط Associativity

• للعوامل ذات الأسبقية نفسها: التقييم من اليسار إلى اليمين.

$$1 - 3 - 5$$

يطرح ١ - ٣ ثم يطرح الناتج من ٥ #

$$1 + 2 ** 4$$

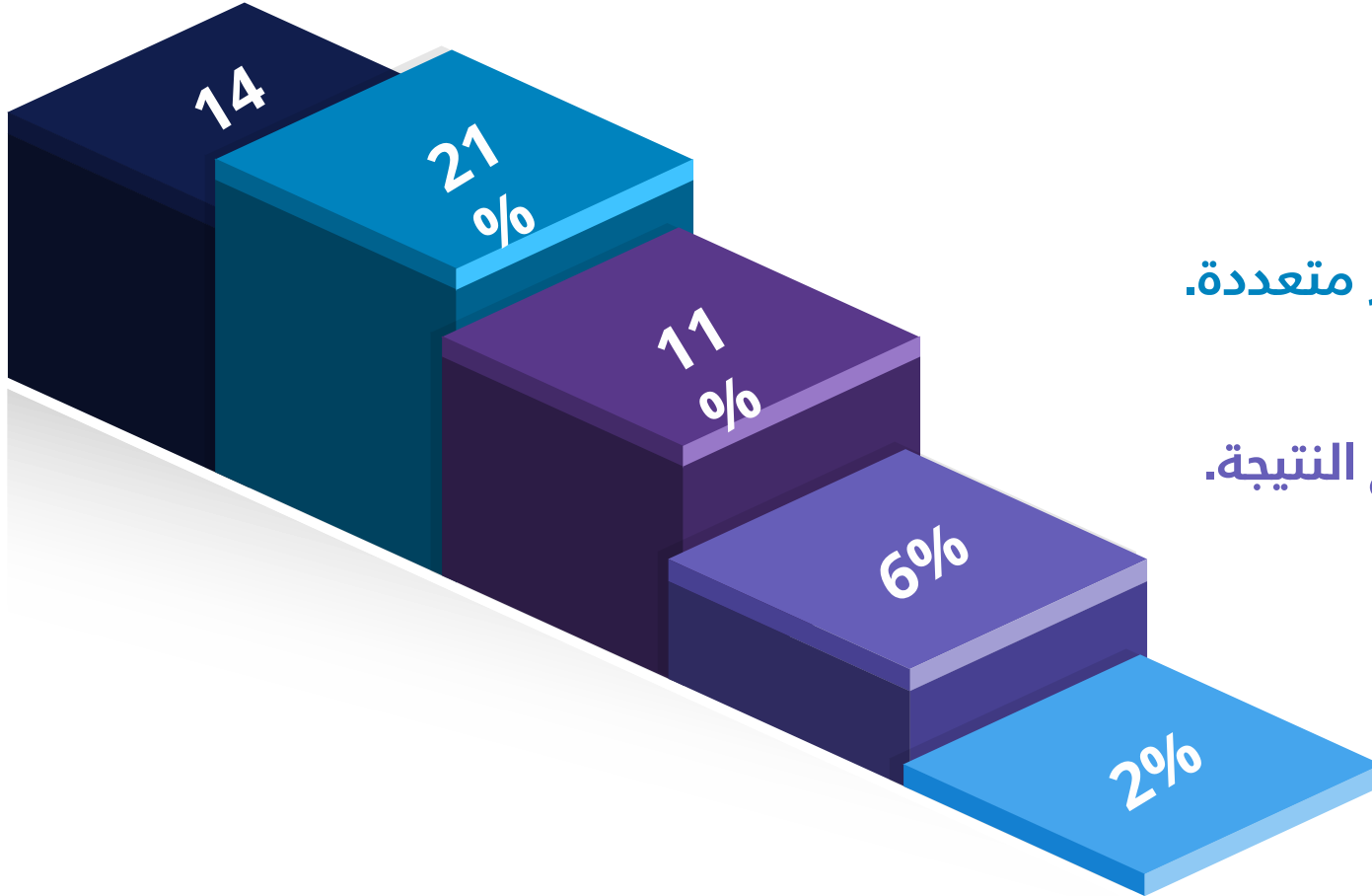
الرفع بالأس قبل الجمع #

$$٢ * ٣ ** ٣$$

الرفع بالأس قبل الضرب #



الموضوع الثالث | نصائح لكتابة تعابير واضحة



أضف أقواسًا عند الشك.



افصل الخطوات المعقّدة إلى سطور متعددة.



اختبر التعابير على قيم بسيطة لتوقع النتيجة.



• العامل + يعني الضم وليس الجمع العددي.

العامل * يكرّر السلسلة عددًا من المرات.

```
'Test ' * 3 # 'Test Test Test'
```



الموضوع الثالث | التبدل بين الأنواع عند الحاجة

• لا يمكن ضمّ عدد وسلسلة مباشرة؛ حوّل النوع أولًا:

```
name = 'STD-'  
num = 15  
name + str(num) # 'STD-15'
```



الموضوع الثالث | العوامل المنطقية



-تعمل على تعابير and و or و not

-تعطي قيمًا منطقية.

-تُستخدم لبناء شروط مركبة داخل التعابير.

```
x > 0 and x < 10
#الناتج يكون true or false
n % 2 == 0 or n % 3 == 0
#يقبل القسمة على 2 أو 3
not (x > y)
#نفي الشرط
```





• شائع فحص شروط تعتمد على نتائج حسابية:

```
score = 75  
passed = (score >= 60) and (score <= 100)  
#passed is true
```



الموضوع الثالث | مثال تطبيقي (١) — تحويل الثواني

مطلوب

- تحويل total_seconds إلى ساعات/دقائق/ثوانٍ.



```
total_seconds = 7385 # مثال

hours = total_seconds // 3600
minutes = (total_seconds % 3600) // 60
seconds = total_seconds % 60

print(hours, minutes, seconds) # 2 3 5
print(f"{hours:02d}:{minutes:02d}:{seconds:02d}")
# 02:03:05
```



الموضوع الثالث | مثال تطبيقي (٢) — استخراج الأرقام

مطلوب

: استخراج الرقم/الأرقام الأخيرة وهل العدد يقبل القسمة على خمسة.

```
x = 987654  
last_digit = x % 10 # 4  
last_two = x % 100 # 54  
is_multiple5 = (x % 5 == 0)
```



الموضوع الثالث | مثال تطبيقي (٣) — التعبيرات النصية

مثال : للتعبيرات النصية

```
first_name = 'Ali'  
msg = ('Hi ' + first_name + '! ')  
      * 2  
print(msg) # Hi Ali! Hi Ali!
```



الموضوع الثالث | نصائح للبرمجة الجيدة

أفضل الممارسات:

١. استخدم الأقواس للوضوح:

result = (a + b) * (c - d) # جيد
result = a + b * c - d # أقل وضوحاً

٢. قسم التعبيرات المعقدة

result = ((a + b) * c) / ((d - e) ** 2) # معقد
أوضح
sum_ab = a + b
diff_de = d - e
result = (sum_ab * c) / (diff_de ** 2)

٣. استخدم أسماء متغيرات واضحة



برنامج حساب تكلفة رحلة

```
distance = float(input("أدخل المسافة بالكيلومتر "))
fuel_price = float(input("أدخل سعر اللتر "))
consumption = float(input("أدخل استهلاك الوقود  
(كم/100 لتر) "))
```

حساب التكلفة

```
fuel_needed = (distance * consumption) / 100
total_cost = fuel_needed * fuel_price
```

```
print(f"لتر {fuel_needed:.2f}: الوقود المطلوب")
print(f"ريال {total_cost:.2f}: التكلفة الإجمالية")
```

حساب التكلفة لكل كيلومتر

```
cost_per_km = total_cost / distance
print(f"ريال {cost_per_km:.2f}: التكلفة لكل كيلومتر")
```

اكتب برنامج لحساب تكلفة الرحلة بحيث
المستخدم يدخل المسافة بالكيلومتر
ويدخل سعر اللتر ويدخل استهلاك
الوقود ثم يحسب التكلفة.



الموضوع الثالث | اكتب برنامج لحساب مساحة ومحيط دائرة

اكتب برنامج لحساب مساحة ومحيط الدائرة

```
import math

radius = float(input("أدخل نصف القطر: "))
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"المساحة: {area:.2f}")
print(f"المحيط: {circumference:.2f}")
```



الموضوع الثالث | أمثلة عملية على باقي القسمة



التحقق من الأرقام الزوجية والفردية

x = 12

if x % 2 == 0:

print("زوجي")

else:

print("فردى")

استخراج آخر رقم

number = 1234

last_digit = number % 10

print(last_digit) # النتيجة: 4

استخراج آخر رقمين

last_two = number % 100

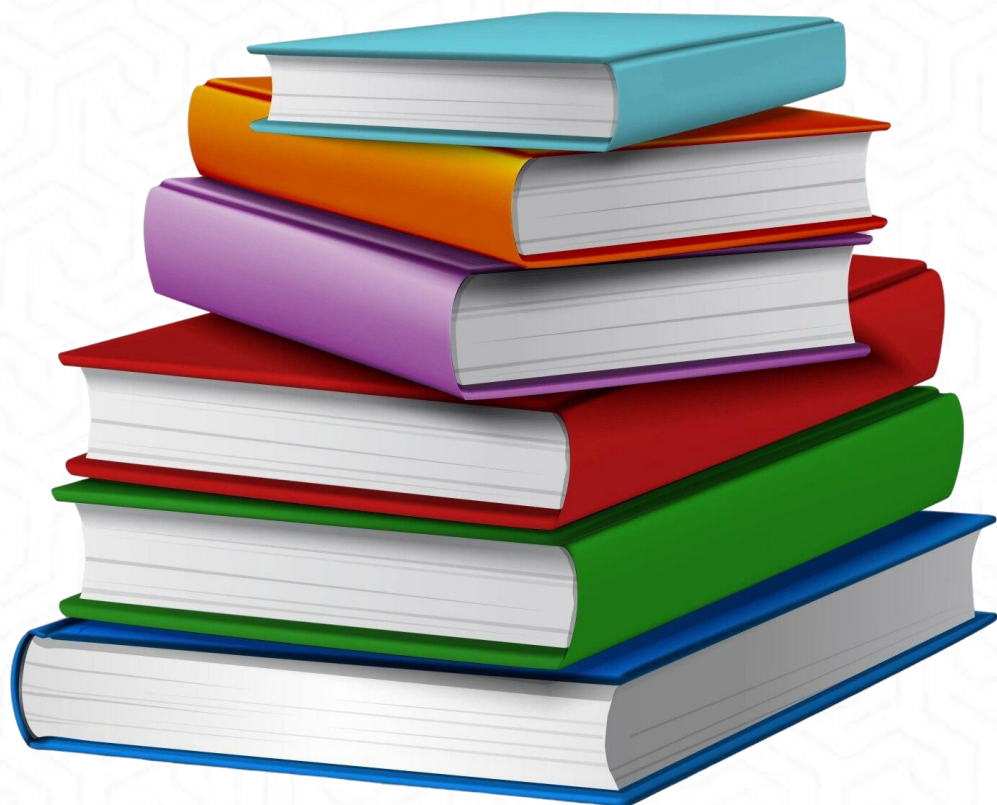
print(last_two) # النتيجة: 34



١. اكتب تعبيرًا يتحقق من كون n يقبل القسمة على ٣ و ٥.
٢. أعد صياغة تعبير معقد باستخدام أقواس لتسهيل قراءته.
٣. أنشئ سلسلة نصية من جزأين وكررها ٤ مرات في سطر واحد.



Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.
الفصل الثالث





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: : PYT103

الموضوع: المتغيرات وأنواع البيانات



الموضوع الثاني: المتغيرات، أنواع البيانات، وإدخال/إخراج أساسي

الأهداف

في نهاية هذا الدرس ستكون قادرًا على أن:

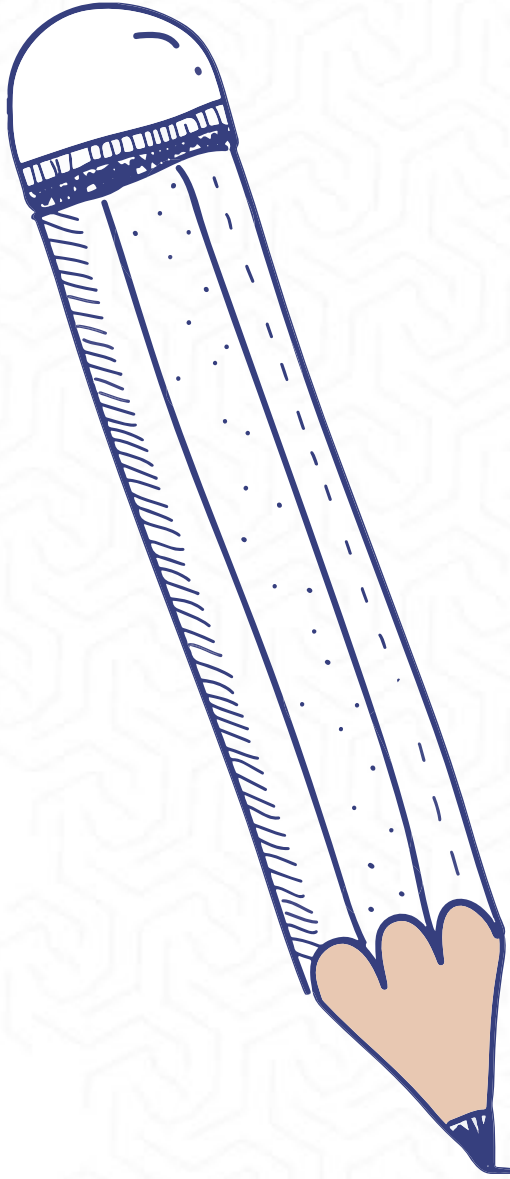
- ✓ يعرف المتغيرات وأنواع البيانات في بايثون.
- ✓ ينفذ العمليات الأساسية على البيانات.
- ✓ يستخدم الدوال المدمجة مثل `print` و `input`.
- ✓ يطبق ما تعلمه من خلال أمثلة عملية وتطبيقات.



وصف الدرس

في هذا الدرس سنتناول:

- التعرّف على مفهوم المتغيّرات في لغة بايثون.
- التعرّف على أنواع البيانات المختلفة في بايثون: الأنواع العددية - السلاسل النصية.
- قراءة الإدخال من المستخدم.
- عرض النتائج على الشاشة باستخدام الدوال المدمجة.
- تطبيق أمثلة عملية لتوضيح المفاهيم.



الموضوع الثاني | المتغيرات؟

المتغيرات:



- المتغير هو اسم يربط قيمة معينة في الذاكرة.
- تُستخدم المتغيرات لتخزين البيانات واستخدامها في العمليات.
- يمكن تغيير قيمة المتغير بإعادة الإسناد بقيمة أخرى أثناء تنفيذ البرنامج.

نلاحظ في المثال ثلاث عمليات إسناد لمتغيرات:



الأولى إسناد سلسلة نصية إلى متغير جديد يُسمى `message`،
أما الثانية، فإسناد العدد الصحيح ٤٢ للمتغير `count`، أما الثالثة،
فإسناد القيمة التقريبية لـ π للمتغير `pi`

لإظهار قيمة المتغير بإمكانك استخدام تعليمة الطباعة

`print`

```
message = "مرحبا"  
count = 42  
pi = 3.14
```

الموضوع الثاني | قواعد تسمية المتغيرات



يجب أن يبدأ الاسم للمتغير بحرف.



يمكن أن يحتوي على حروف وأرقام وشرطات سفلية



يسمح بوجود رمز الشرطة السفلية _ داخل اسم المتغير وتستخدم غالبًا في أسماء المتغيرات التي تحوي على العديد من الكلمات، مثل: `my_name`، أو `air_speed_of_unladen_swallow`



يمكن لأسماء المتغيرات أن تكون ذات أطوال مختلفة.



من الأفضل أن يبدأ اسم المتغير بحروف صغيرة (سنرى السبب لاحقًا).



الأسماء حساسة لحالة الأحرف X يختلف عن x.



لا تستخدم الكلمات المحبوزة في لغة البايثون أو الكلمات المفتاحية كأسماء متغيرات.



اختر أسماء ذات معنى لسهولة الفهم.



من الممكن أن تبدأ أسماء المتغيرات بالشرطة السفلية _ ، لكن بشكل عام نتجنب ذلك إن لم نكن نكتب شيفرة لمكتبة برمجية قد يستخدمها الآخرون.



الموضوع الثاني | أمثلة على تسمية المتغيرات والكلمات المفتاحية؟

```
# أسماء صحيحة #  
my_name = "أحمد"  
age = 25  
_private = "سري"  
camelCase = "جيد"
```

٣٥ كلمة مفتاحية يجب عدم استخدامها كمتغيرات:

and	del	from	None	True
as	elif	global	nonlocal	try
assert	else	if	not	while
break	except	import	or	with
class	False	in	pass	yield
continue	finally	is	raise	async
def	for	lambda	return	await

```
# أسماء خاطئة #  
٢name = " # يبدأ برقم  
class = " # كلمة محبوزة  
animal@= @ # يحتوي على رموز مثل  
my-name = " # يحتوي على شرطة"
```

يمكن تغيير قيمة المتغير عن طريق إسناد جديد



تُقيّم الجهة اليمنى أولاً ثم تُخزن النتيجة



يؤدي تحديث متغير غير معرف إلى حدوث خطأ من نوع `NameError`.



```
count = 0
count = count + 1
unknown = unknown + 1 # يثير خطأ
```


النوع	الوصف	مثال
int	الأعداد الصحيحة	x = 42
float	الأعداد العشرية	y = 3.14
str	السلاسل النصية	name = "محمد"

التحقق من نوع البيانات:

```
x = 42
type(x)
<class 'int'>

y = 3.14
type(y)
<class 'float'>

name = "سارة"
type(name)
<class 'str'>
```

الموضوع الثاني | أنواع البيانات



هياكل بيانات
`list ,tuple ,set ,dict`



نوع نصي
`str`



نوع منطقي
`bool`



أنواع عددية
`int ,float ,complex`

الموضوع الثاني | متغيرات الأنواع العددية

int: أعداد صحيحة غير محدودة الحجم



float: أعداد عشرية بدقة مزدوجة



complex: أعداد مركبة لها جزء حقيقي
وتخيلي



يتم تحويل الأنواع تلقائياً عند
العمليات المشتركة.



```
a = 5
b = 2.0
c = 3 + 4j
d = a + b # float النتيجة متغير من نوع
```

خصائص الأعداد الصحيحة:

أعداد بدون فاصلة عشرية.



يمكن أن تكون موجبة أو سالبة.



لا يوجد حد أقصى للحجم في Python.



```
# أعداد صحيحة
age = 25
temperature = -10
big_number = 10000000000000

# طباعة القيم
print(age)          # 25
print(temperature)  # -10
print(type(age))    # <class 'int'>

# عمليات حسابية
x = 10
y = 3
print(x + y)        # 13
print(x - y)        # 7
print(x * y)        # 30
print(x // y)       # 3 (القسمة الصحيحة)
```

خصائص الأعداد العشرية:

```
# أعداد عشرية
price = 15.99
height = 1.75
scientific = 2.5e3 # 2500.0

# طباعة القيم
print(price) # 15.99
print(scientific) # 2500.0
print(type(price)) # <class 'float'>

# عمليات حسابية
x = 10.5
y = 2.5
print(x + y) # 13.0
print(x / y) # 4.2
print(x ** 2) # 110.25 (الأس)
```

أعداد تحتوي على فاصلة عشرية



تُستخدم للحسابات الدقيقة



يمكن كتابتها بالترميز العلمي



خصائص السلاسل النصية:

- تُحاط بعلامات اقتباس مفردة ' أو مزدوجة " .
- يمكن أن تحتوي على أي محارف.
- غير قابلة للتعديل immutable
- السلسلة النصية عبارة عن تسلسل من الأحرف.
- يمكن دمج السلاسل باستخدام + وتكرارها باستخدام *
- يمكن الوصول إلى الأحرف عن طريق الفهرسة.

```
# سلاسل نصية
name = "أحمد محمد"
city = 'الرياض'
empty_string = ""

# طباعة القيم
print(name)          # أحمد محمد
print(type(name))    # <class 'str'>

# دمج السلاسل
first_name = "سارة"
last_name = "أحمد"
full_name = first_name + " " + last_name
print(full_name)     # سارة أحمد

# تكرار السلاسل
greeting = "مرحباً" * 3
print(greeting)      # مرحباً مرحباً مرحباً
```

```
# تحويل من نص إلى رقم
age_str = "25"
age_int = int(age_str)
print(age_int)          # 25
print(type(age_int))    # <class 'int'>
```

```
# تحويل من نص إلى عدد عشري
price_str = "15.99"
price_float = float(price_str)
print(price_float)      # 15.99
```

```
# تحويل من رقم إلى نص
number = 42
number_str = str(number)
print(number_str)       # "42"
print(type(number_str)) # <class 'str'>
```

```
# أمثلة أخرى
print(int(3.14))        # 3 (يحذف الجزء العشري)
print(float(42))        # 42.0
```

- int() تحويل إلى عدد صحيح



- float() تحويل إلى عدد عشري



- str() تحويل إلى سلسلة نصية



التابع input():

```
# إدخال بسيط
name = input(" ما اسمك؟ ")
print("مرحباً" + name)

# إدخال مع تحويل النوع
age_str = input(" كم عمرك؟ ")
age = int(age_str)
print("سنة", age, "عمرك هو")

# طريقة مختصرة
age = int(input(" كم عمرك؟ "))
height = float(input(" كم طولك؟ "))

# مثال متكامل
name = input(" أدخل اسمك: ")
age = int(input(" أدخل عمرك: "))
print(f"سنة {age} وعمرك {name} اسمك")
```

يستقبل البيانات من المستخدم



دائماً يُرجع نوع str



يمكن إضافة رسالة توضيحية



أمثلة استخدامات مختلفة للطباعة:

```
# طباعة بسيطة
print("مرحباً بالعالم")

# طباعة متغيرات
name = "أحمد"
age = 25
print(name)
print(age)

# طباعة متعددة في سطر واحد
print("العمر:", name, "الاسم:", age)

# استخدام الفاصل المخصص
print("الموز", "البرتقال", "التفاح", sep="-")
# الموز-البرتقال-التفاح: النتيجة

# تغيير نهاية السطر
print("السطر الأول", end=" ")
print("السطر الثاني")
# السطر الأول السطر الثاني: النتيجة
```

مثال:

تطبيقي شامل - حاسبة بسيطة البرنامج يقوم بإدخال عددين من المستخدم ثم يقول بعملية جمع وطرح وضرب وقسمة للعددين ويعرض النتائج على الشاشة.

```
print("=== حاسبة بسيطة ===")

# إدخال البيانات
num1 = float(input("أدخل الرقم الأول: "))
num2 = float(input("أدخل الرقم الثاني: "))

# العمليات الحسابية
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2

# إظهار النتائج
print(f"\nالنتائج:")
print(f"{num1} + {num2} = {addition}")
print(f"{num1} - {num2} = {subtraction}")
print(f"{num1} × {num2} = {multiplication}")
print(f"{num1} ÷ {num2} = {division:.2f}")

# معلومات إضافية
print(f"\nنوع البيانات:")
print(f"نوع الرقم الأول: {type(num1)}")
print(f"نوع النتيجة: {type(addition)}")
```


مثال:

```
print("=== حاسبة مؤشر كتلة الجسم ===")

# إدخال البيانات الشخصية
name = input("أدخل اسمك: ")
weight = float(input("أدخل وزنك بالكيلوغرام: "))
height = float(input("أدخل طولك بالمتر: "))

# حساب مؤشر كتلة الجسم
bmi = weight / (height ** 2)

# تحديد الفئة
if bmi < 18.5:
    category = "نقص في الوزن"
elif bmi < 25:
    category = "وزن طبيعي"
elif bmi < 30:
    category = "زيادة في الوزن"
else:
    category = "سمنة"

# إظهار النتائج
print(f"\n=== النتائج لـ {name} ===")
print(f"الوزن: {weight} كغ")
print(f"الطول: {height} م")
print(f"مؤشر كتلة الجسم: {bmi:.2f}")
print(f"الفئة: {category}")
```

: تطبيقي شامل - حساب مؤشر كتلة الجسم يقوم بإدخال الطول والوزن من المستخدم ثم يقول بعملية حساب $bmi = weight / (height ** 2)$ إذا كانت قيمته أقل من 18,5 يطبع عبارة "نقص في الوزن" أما إذا كان بين 25 و 18,5 يطبع "وزن طبيعي" أما إذا كانت القيمة بين 3 و 25 يطبع "زيادة في الوزن" إذا أعلى من ذلك يطبع "سمنة".

١. خطأ في نوع البيانات:

```
# خطأ شائع
age = input("كم عمرك؟")
next_year = age + 1 # خطأ!
# لا يمكن جمع قيمة نص ورقم

# الحل الصحيح
age = int(input("كم عمرك؟"))
next_year = age + 1
print(f"سنة {next_year} القادم ستكون")
```

٢. خطأ في تحويل النوع:

```
# خطأ محتمل
user_input = input("أدخل رقماً: ") # إذا أدخل "abc" المستخدم
number = int(user_input) # ValueError!

# سنتعلمه لاحقاً حل أفضل
try:
    number = int(input("أدخل رقماً: "))
    print(f"الرقم هو: {number}")
except ValueError:
    print("يجب إدخال رقم صحيح: خطأ")
```

التمرين الأول: طباعة معلومات شخصية

اكتب برنامجاً يطلب من المستخدم:

- الاسم الأول والأخير
- العمر

• المدينة

ثم يطبع جملة تعريفية

```
first_name = input("أدخل اسمك الأول: ")
last_name = input("أدخل اسمك الأخير: ")
age = int(input("أدخل عمرك: "))
city = input("أدخل مدينتك: ")

print(f"اسمي {first_name} {last_name}! مرحباً")
print(f"سنة {city} وأسكن في {age} عمري")
```

التمرين الثاني: تحويل العملات

اكتب برنامجاً يحول من الريال
السعودي إلى الدولار الأمريكي
(سعر الصرف: ١ دولار = ٣,٧٥ ريال)

```
sar_amount = float(input("أدخل المبلغ بالريال "))
exchange_rate = 3.75
usd_amount = sar_amount / exchange_rate

print(f"{sar_amount} ريال = {usd_amount:.2f}  
      ("دولار
```


اختبر فهمك:

1. float و int ما الفرق بين.
2. أحياناً؟ input() لماذا نحتاج لتحويل نتيجة.
3. print(type("123")): ما نتيجة تنفيذ.
4. كيف نطبع متغيرين في سطر واحد؟.
5. 123name=: ما الخطأ في.

الإجابات:

1. للأعداد العشرية float للأعداد الصحيحة، int.
2. حتى لو أدخل المستخدم رقماً str دائماً يُرجع input() لأن.
3. <class 'str'>
4. F-strings أو print(var1, var2)
5. اسم المتغير لا يمكن أن يبدأ برقم.

Python for Everybody: Exploring Data in Python 3 Book by Charles Severance. الفصل الثاني





شكراً لكم



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: : PYT103

الموضوع: مقدمة في البرمجة وبايثون



في نهاية هذا الدرس سيكون الطالب قادراً على أن:

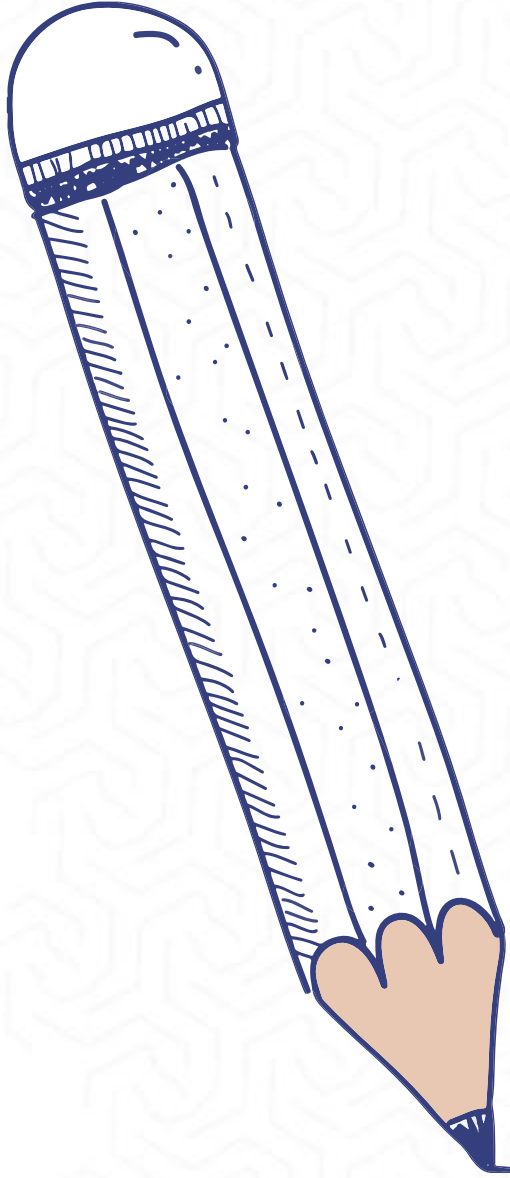
- ✓ يعرف مكونات الحاسوب الرئيسية ((CPU، الذاكرة، أجهزة الإدخال والإخراج، الشبكة، ويحدد دور كل مكون.
- ✓ يميز بين المفسر والمترجم في البرمجة.
- ✓ يشغل المفسر التفاعلي للبايثون وينفذ جملة طباعة باستخدام `print()`.
- ✓ يكتب برنامجاً بسيطاً بلغة بايثون وينفذه من ملف سكريبت (`py`).
- ✓ يطبق أنماط بناء البرامج الأساسية (التسلسل، الإدخال/الإخراج، الشرط، التكرار، والدوال).
- ✓ يشخص الأخطاء الشائعة في بايثون (مثل `SyntaxError`, `NameError`) ويستخدم أساليب أولية للتنقيح (مثل الطباعة للفحص).



الموضوع الأول | وصف الدرس

في هذا الدرس سنتناول:

- مدخل عملي إلى التفكير البرمجي باستخدام لغة بايثون. يبدأ بالتعرّف على مكوّنات الحاسوب وكيف «نفكّر كمبرمجين»، ثم ينتقل إلى تشغيل المفسّر التفاعلي وتنفيذ أول برنامج. Hello, World!
- كتابة سكربت بسيط وتشغيله من الطرفية.
- أنماط بناء البرامج الأساسية وآلية تفاعل البرنامج مع المستخدم عبر `input()`.
- التركيز على فهم الرسائل الخطأ الأكثر شيوعًا وتبني ممارسات تنقيح سريعة تساعد الطالب على بناء الثقة أثناء التعلم.



الموضوع الأول | ما هي مكونات الحاسب؟



استقبال المدخلات (Input)

- يتلقّى الحاسب بيانات وأوامر من لوحة المفاتيح، الفأرة، الميكروفون، شاشة اللمس... إلخ.
- كما يمكن أن يأتي المدخل من ملفات مخزنة أو عبر الشبكة.

المعالجة (Processing)

- وحدة المعالجة المركزية (CPU) تنفّذ التعليمات بسرعة عالية جدًا (مليارات المرات في الثانية)، وهي قلب تنفيذ أوامر البرامج.

التخزين والاسترجاع (Memory & Storage)

- الذاكرة الرئيسية: سريعة وقريبة من المعالج، لكن محتواها يزول بانقطاع الطاقة.
- الذاكرة الثانوية: أبداً ولكنها تحتفظ بالبيانات والبرامج عند إيقاف التشغيل (أقراص، ذاكرة وميضية، USB...).

المخرجات (Output)

- يعرض النتائج على الشاشة، ويُسّغّل الصوت، ويكتب إلى ملفات... وهي وسائل تفاعل المستخدم مع ما أنجزه البرنامج.

الاتصال والتواصل (Networking)

- معظم الحواسيب متّصلة بالشبكات لتبادل البيانات والموارد؛ ويمكن النظر إلى الشبكة كامتدادٍ للتخزين/النقل وإن كان أبداً وأقلّ أماناً من التخزين المحلي.

التنسيق بين المكونات عبر البرامج

- وظيفة المبرمج هي توجيه الحاسب لاستخدام هذه الأجزاء معًا لحلّ مشكلة: إعطاء أوامر للمعالج، والقراءة/الكتابة من الذاكرة الرئيسية أو الثانوية، والتعامل مع الشبكة وأجهزة المدخلات/المخرجات.



الموضوع الأول | ما المهام التي يقوم بها الحاسب؟

معالجة و إنتاج
الوسائط المتعددة.

تخزين المعلومات
و إعادة استخدامها.

العمليات الحسابية
و المنطقية.



الموضوع الأول | مفهوم لغات البرمجة



50
%

الخوارزميات هي: سلسلة
من الخطوات التي يمكن
اتباعها لإيجاد الحل
المطلوب.

81
%

فالحاسب لكي يفهم
المطلوب منه فإنه يجب على
المبرمج أن يكتب الحلول
بطريقة مفصلة و واضحة و
دقيقة جداً ليتمكن الحاسب
من فهمها والتعامل معها و
ذلك عن طريق (الخوارزميات).

الموضوع الأول | أهمية لغات البرمجة

الحاسب الآلي جهاز لا يفهم إلا لغة واحدة
للتعامل معه و هي (لغة الآلة)



المكونة من رمزين فقط هي (1 , 0)



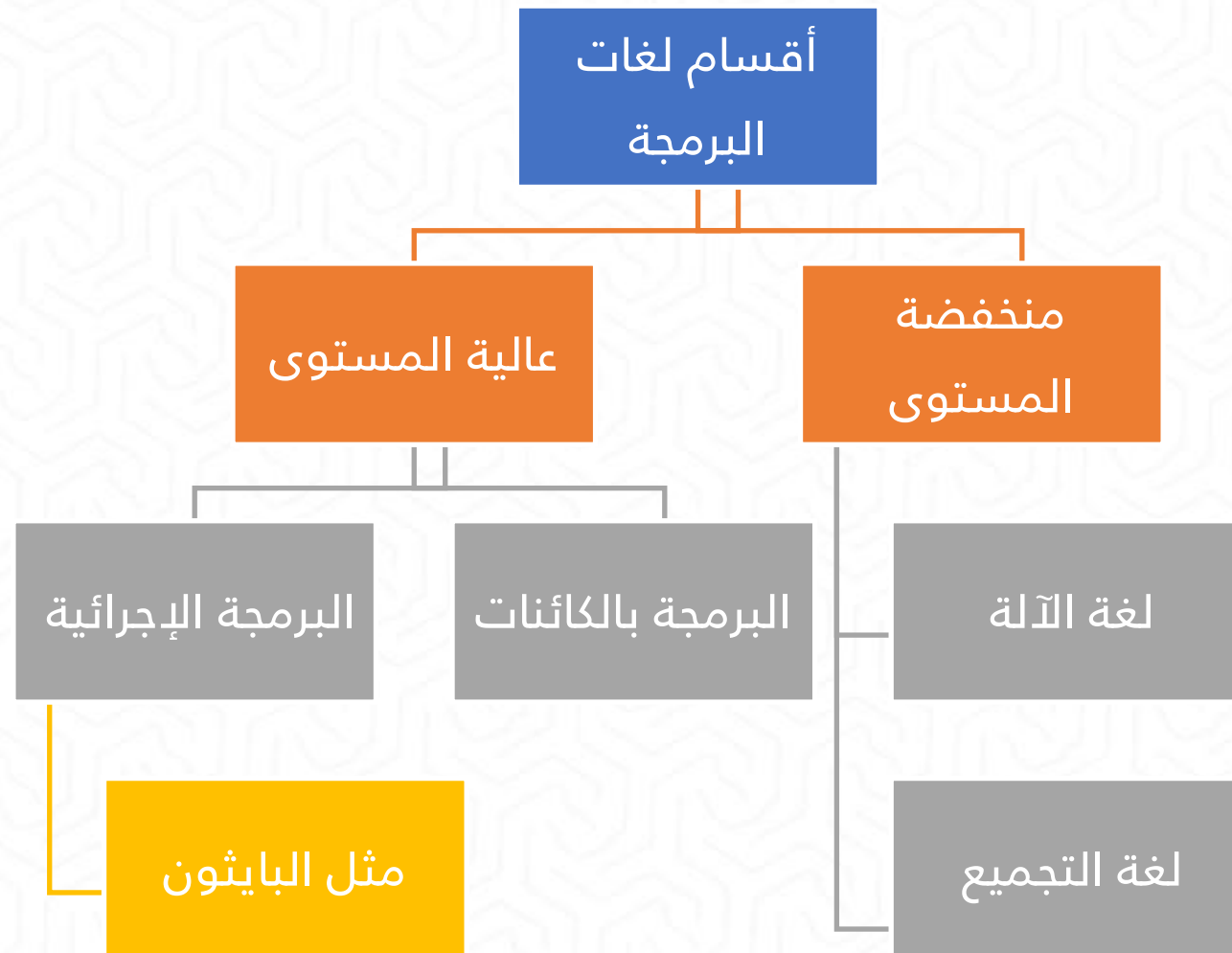
لذلك نقوم باستخدام لغات البرمجة كوسيلة
لتبسيط التواصل بين المبرمج و بين الحاسب
الآلي.



وتختلف كل لغة عن الأخرى، إلا أن البايثون
من اللغات البسيطة حتى أنها أبسط
من (C++ , JAVA)



الموضوع الأول | أقسام لغات البرمجة



الموضوع الأول | أقسام لغات البرمجة

البرمجة بالكائنات

هذا النص هو مثال لنص يمكن أن يستبدل في نفس المساحة، لقد تم توليد هذا النص من مولد النص العربي.

لغات البرمجة منخفضة المستوى

قريبة من العتاد وتمنح تحكمًا دقيقًا بالذاكرة والمسجلات، لكنها أصعب قراءة وأقل قابلية للنقل بين المعماريات.

لغة الآلة

تعليمات ثنائية (0/1) يفهمها المعالج مباشرة. أدنى مستوى من التجريد، سريعة جدًا لكن صعبة على البشر ومعمارية-المعالج تحددها.

لغة التجميع (Assembly)

تمثيل رمزي قصير للتعليمات الثنائية (مثل: MOV, ADD). علاقة شبه واحد-لواحد مع لغة الآلة، ويحوّلها "مجّمع" (Assembler) إلى ثنائيات. تمنح تحكمًا عاليًا لكنها مرتبطة بمعمارية محددة.

لغات البرمجة عالية المستوى

لغات قريبة من طريقة تفكير الإنسان وتوفّر تجريدًا عن تفاصيل العتاد (الذاكرة، المسجلات...). يسهل قراءتها وكتابتها وهي غالبًا منقولة بين الأنظمة. أمثلة: Python, Java, C#, JavaScript.

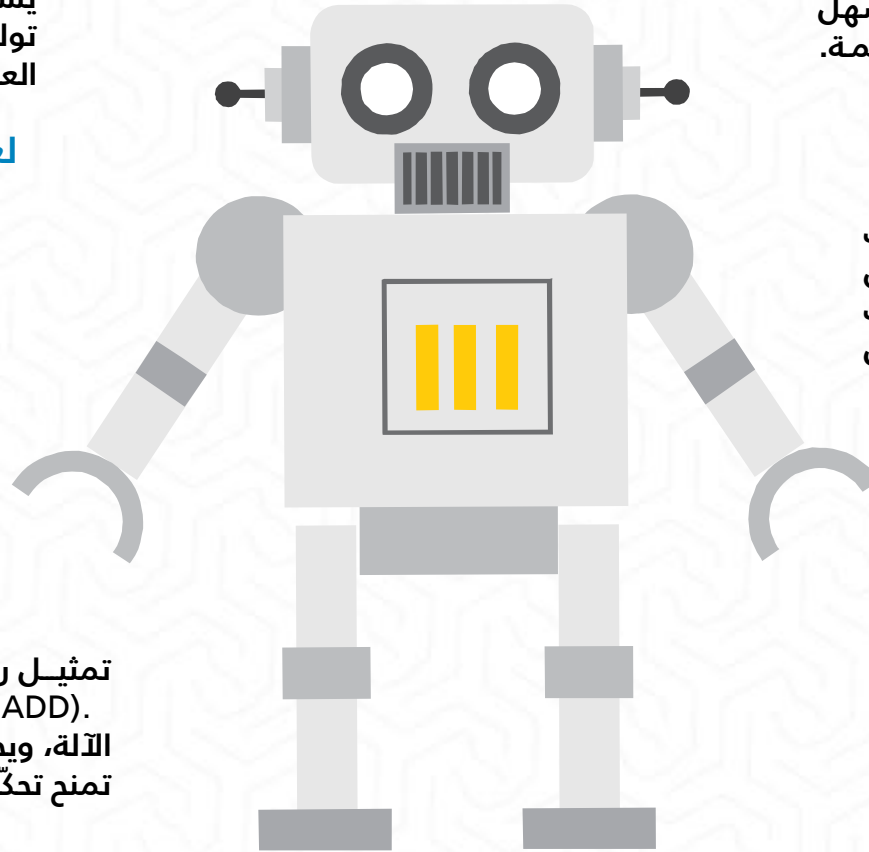
البرمجة الإجرائية

أسلوب يبنى البرنامج كسلسلة خطوات/إجراءات (Functions) تستدعى لمعالجة بيانات. التركيز على "ماذا نفعل أولاً ثم ثانيًا...". مناسبة للمشكلات الخوارزمية المباشرة. أمثلة شائعة: C, وبايثون تدعم هذا النمط.

البرمجة بالكائنات

نمط ينظّم الشيفرة حول "كائنات" تجمع البيانات مع العمليات عليها. يعتمد مبادئ: التغليف، الوراثة، التعدّد الشكلي. مناسب للأنظمة الكبيرة وقابلية إعادة الاستخدام. أمثلة

Java, C++,



الموضوع الأول | المكوّنات الأساسية للبرامج



الموضوع الأول | المفسر والمترجم



لا تفهم وحدة المعالجة المركزية (CPU) أيًا من هذه اللغات، بل تفهم فقط لغة واحدة ندعوها لغة الآلة، وهي لغة بسيطة جدًا كالآتي:



مكونة من واحد وصفر، لكن كلمة "بسيطة" هنا قد تكون خادعة، فهي بسيطة في بنيتها، إذ إنها تتألف من 0 و1 ولكن ستجد أنها صعبة ومعقدة جدًا حين تحاول أن تكتب برنامجًا بواسطتها، لذا يرمج قليل من المبرمجين بلغة الآلة ولأغراض محدّدة.



بايثون لغة عالية المستوى، أي أنها صممت لتكون واضحة نسبيًا للبشر، وللحواسيب في الوقت نفسه لتقرأها وتعالجها، ومن اللغات عالية المستوى أيضًا: C++, PHP, Basic, JavaScript, Perl, Ruby وغيرها الكثير.



```
00101000111010010010101000
0001111
11100110000011101010010101
...101101
```



الموضوع الأول | المفسّر والمترجم

أنشأت العديد من المترجمات حتى نستطيع البرمجة بلغات عالية المستوى كبايثون وجافا سكربت، وتحول هذه المترجمات تلك البرامج إلى لغة الآلة حيث تنفذها وحدة المعالجة (CPU).

ما الفرق بين المفسّر والمترجم؟

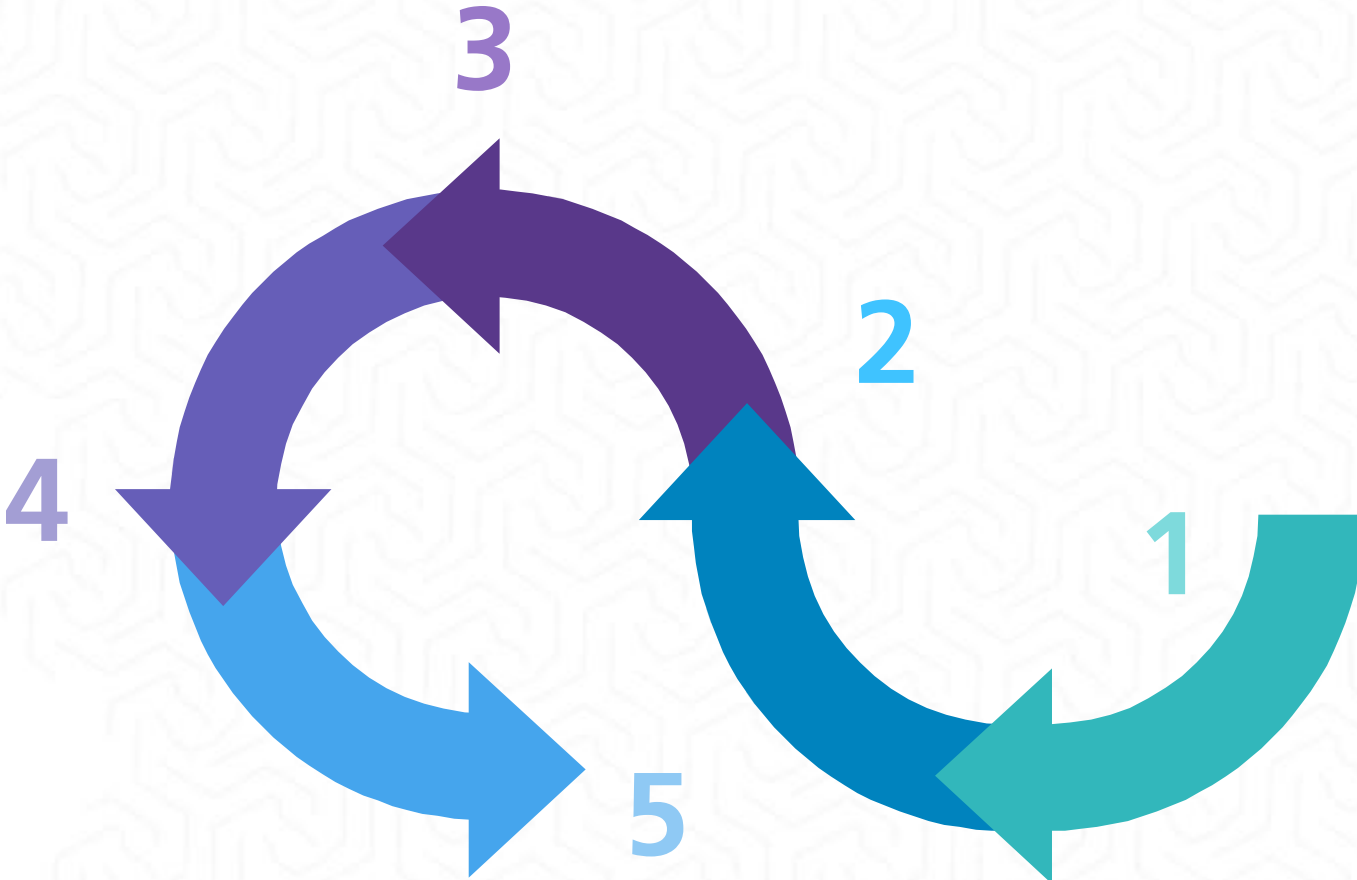
المفسّر: ينفّذ الكود سطرًا بسطر مباشرة (مثل مفسّر بايثون التفاعلي).



المترجم: يحوّل البرنامج كاملاً إلى كود برنامج بلغة الآلة قبل التشغيل.



بايثون نفسها مكتوبة بلغة C و موزعة كبرنامج مترجم يعمل على جهازك.



الموضوع الأول | ماهي بايثون؟



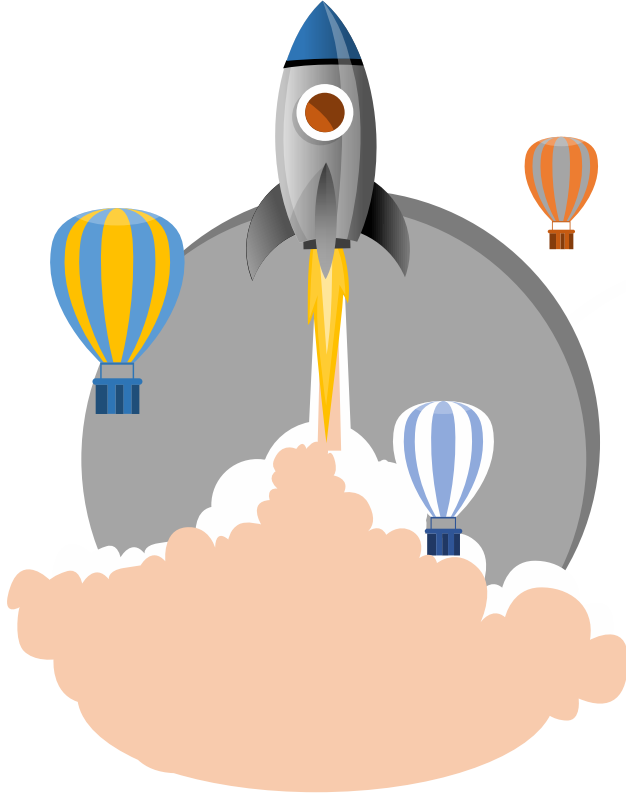
طُورت بايثون في معهد الرياضيات والمعلوماتية الهولندي CWI في مدينة أمستردام على يد جايدو فان روسم في أواخر ثمانينات القرن العشرين، وكان أول إعلان عنها في عام 1991 م.



كُتبت نواة اللغة باستعمال لغة سي. أطلق روسم الاسم "بايثون" على لغته تعبيرًا عن إعجابه بفرقة مسرحية هزلية شهيرة من بريطانيا، كانت تطلق على نفسها اسم مونتي بايثون.



الموضوع الأول | لماذا بايثون؟



لديها مجموعة كبيرة جداً من المراجع.



تعتبر من أبسط اللغات من ناحية القواعد.



قدرات بايثون غير محدودة بالرغم من بساطتها.



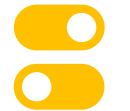
يمكن تعلمها بسرعة.



كتابة البرمجية تتطلب عدد أقل من الأسطر.



تعمل كجسر بين لغات البرمجة الأخرى.



تعمل على جميع أنظمة التشغيل ويندوز - ماك - لينكس بل أنها مضمنة في نظامي ماك و لينكس.



الموضوع الأول | لماذا بايثون؟

01

تعدّ بايثون لغة سهلة الفهم وشيفرتها سهلة القراءة كذلك، وبالتالي فهي أفضل خيار للمبتدئين الذين يريدون تعلّم البرمجة. وهي موجزة جدًا وشيفرتها قصيرة، ما يساهم في زيادة إنتاجيّة المبرمج ويقلّل من نسبة الأخطاء في البرنامج كما يُساعد على إصلاح الأخطاء بسهولة وسرعة.

03

الويب: في أيامنا هذه تعتبر بايثون مع إطار العمل Django من أفضل الأدوات لتطوير مشاريع ضخمة على الويب تتمثّل أساسا في مواقع الأنترنت.

02

تتميّز لغة بايثون بالقوّة والبساطة، فتحوّل لك كتابة برمجيات بسيطة جدًا، كما تمتلك مكتبات متعدّدة تسمح لك بالعمل على مشاريع أكثر تعقيداً.

04

الأنظمة: تعتبر بايثون كذلك من أحسن اللغات التي يستعملها مدراء الأنظمة لإنشاء برمجيات للمساعدة في إنجاز مهام مُتكرّرة وكذلك لصيانة النظام، وإذا كنت ترغب في كتابة تطبيقات Java فإمكانك استخدام بايثون كذلك



الموضوع الأول | أشهر الشركات التي تستخدم بايثون



جوجل Google



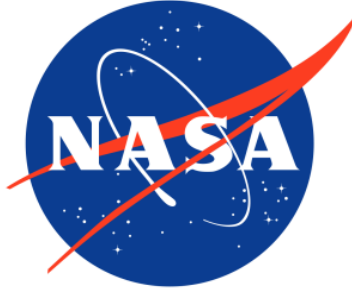
ياهو Yahoo



ناسا NASA



ديزني Disney



• لعبة باتل فيلد Battlefield



الموضوع الأول | تنصيب بايثون



تنصيب بايثون على نظام Windows

يُمكنك تحميل بايثون من الموقع الرسمي.
<https://www.python.org/download/>
إذا تم التحميل بشكل صحيح تجد إضافة في جهازك
تحت اسم IDLE
Integrated Development Environment for
Python
الغرض منها إعطاء وسيلة للتواصل مع لغة البرمجة
بايثون



تنصيب بايثون على نظام Linux أو MacOS

إذا كان لينكس أو ماك أو إس هو نظامك
تشغيلك، فهذا أمر جيّد لأنّ بايثون مُثبّته مُسبقاً
على هاذين النّظامين.



الموضوع الأول | تنصيب بايثون

Colab <https://colab.research.google.com>

يوفر بيئة بايثون جاهزة في المتصفح بدون تثبيت.

إمكانات حوسبة سحابية CPU/GPU/TPU وتشغيل خوارزميات التعلم الآلي بسهولة.

التكامل مع Google Drive:

سجل دخولك بحساب Google ثم افتح:

colab.research.google.com

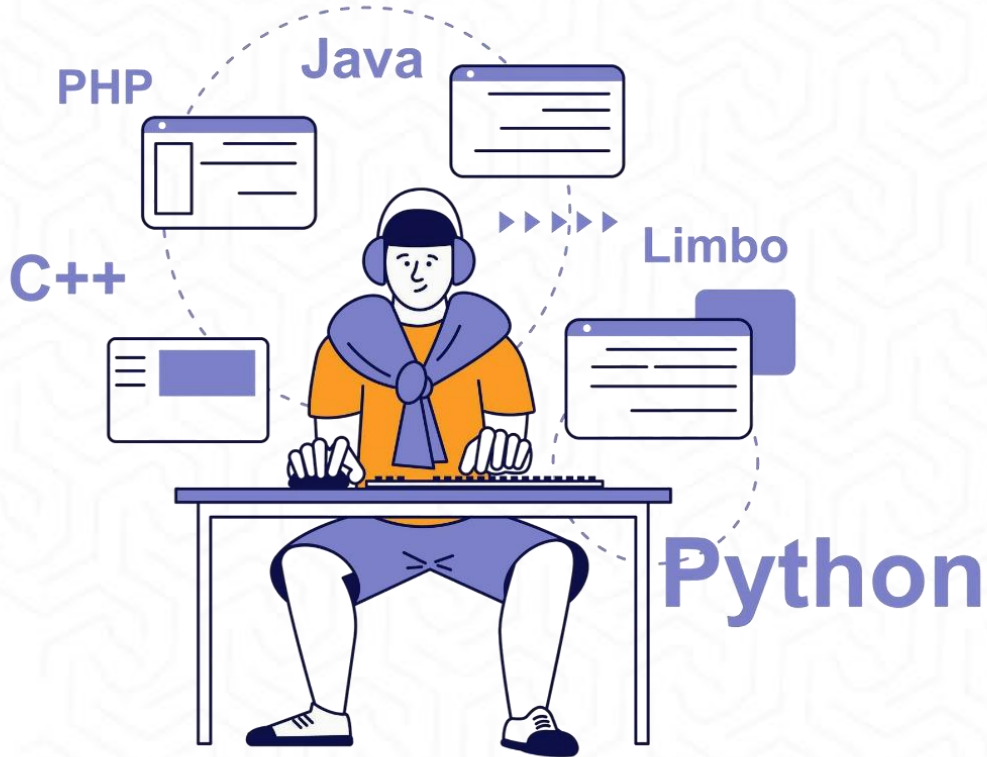
File → New notebook.

اكتب كودك في خلية

Code

واضغط Shift+Enter للتنفيذ

احفظ تلقائيًا (File → Save a copy in Drive)



الموضوع الأول | كتابة برنامج بسيط يحتوي أمر (طباعة نص)

print : دالة مدمجة تطبع مخرجات على الشاشة.
النص بين علامتي اقتباس هو سلسلة نصية String

```
>>> print ('Hello world! ')\nHello world!
```

أين أكتبها؟

1. في المفسر التفاعلي

2. تحفظ كملف سكربت

أنشئ ملفًا اسمه hello.py وضع فيه كود البرمجة



الموضوع الأول | كتابة برنامج بسيط يحتوي أمر (طباعة نص)

: دالة مدمجة تقرأ إدخال المستخدم من لوحة المفاتيح وتُعيده كنص (String).
العبارة بين علامتي الاقتباس هي الرسالة التي تظهر للمستخدم.
يتم حفظ النص في المتغير name.

```
name = input(" ما اسمك؟ ")  
ما اسمك  
> Ali  
print("مرحبًا", name)  
Ali مرحباً
```

أين أكتبها؟

1. في المفسر التفاعلي



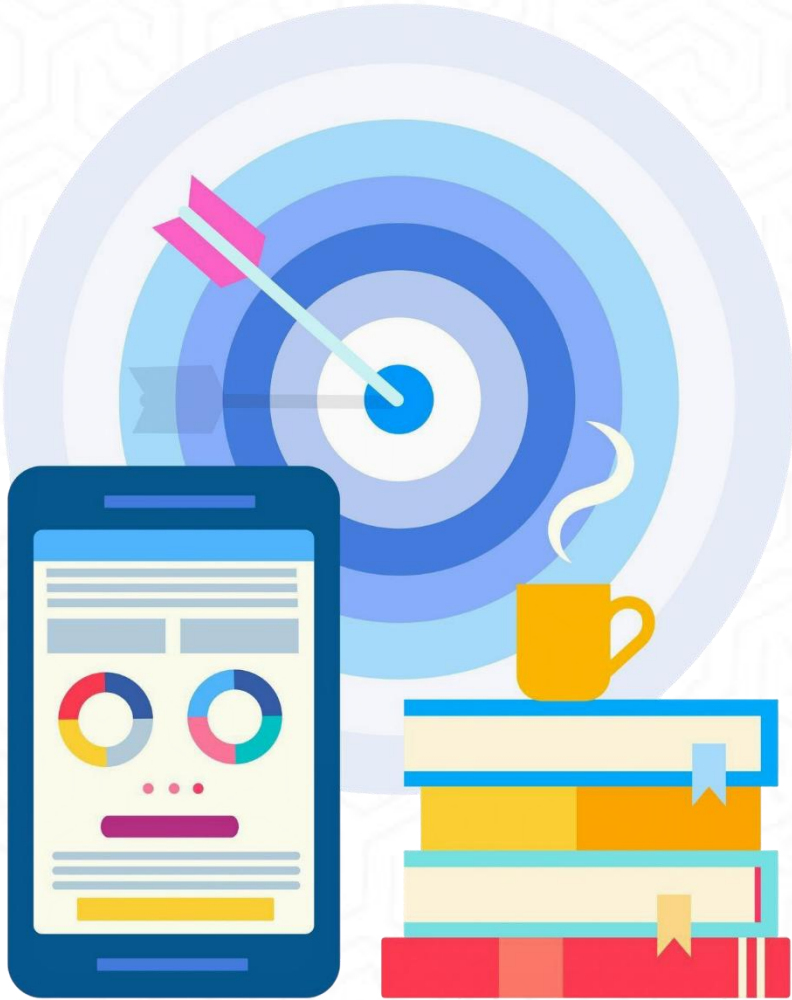
الموضوع الأول | قواعد عامة للبرمجة في بايثون

- لغة البايثون تميز بين الأحرف الكبيرة و الأحرف الصغيرة بايثون تعامل الأسماء التي نستخدمها بتأني سواء كنا نضع هذه الأسماء للمتغيرات، الدول، الكلاسات، الكائنات إلخ.

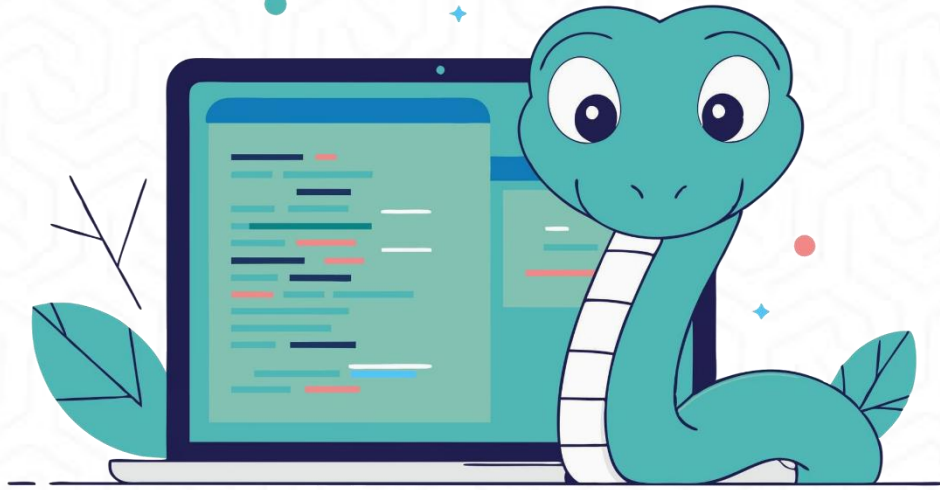
مثال:

Note و note ليسوا شيئاً واحداً.

بينما في لغات برمجة أخرى مثل HTML لا يهم إذا كانت الأحرف كبيرة أو صغيرة.



الموضوع الأول | قواعد عامة للبرمجة في بايثون



كتابة أكثر من أمر واحد على نفس السطر إفتراضياً،
بايثون تعتبر أن كل أمر يكتب على سطر واحد.



إذا أردت كتابة أكثر من أمر على نفس السطر قم
بوضع فاصلة منقوطة بين كل أمرين و هكذا
سيفهم مترجم لغة بايثون أن السطر عليه أكثر من
أمر.



مثال:

```
x = 1 ; y = 2 ; z = 3
```

